Foundation University Islamabad

School of Science and Technology

Name: Hunaina Yasir
Roll Number: 073
Subject: OOP LAB
Section: B

Assignment – 01

Exercise:

Ex 1: Design a multilevel class structure where Person is the base class, Student is derived from Person, and Result is derived from Student. The Result class should include marks of three subjects, and a method to calculate and display the total and average marks. Make sure to use constructor chaining and proper input/output handling in each class.

- **Program:**

```cpp
#include <iostream>
using namespace std;

// Base Class
class Person {
protected:
    string name;
    int age;

public:
    // Constructor of Person
    Person() {
        cout << "Enter Name: ";
        cin >> name;
        cout << "Enter Age: ";
        cin >> age;
    }

    void displayPerson() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
    }
};

```

```cpp
// Derived Class from Person
class Student : public Person {
protected:
    int rollNo;

public:
    // Constructor of Student (Constructor Chaining)
    Student() : Person() {
        cout << "Enter Roll Number: ";
        cin >> rollNo;
    }

    void displayStudent() {
        displayPerson();  // calling base class function
        cout << "Roll Number: " << rollNo << endl;
    }
};

// Derived Class from Student
class Result : public Student {
private:
    int marks1, marks2, marks3;
    int total;
    float average;

public:
    // Constructor of Result (Constructor Chaining)
    Result() : Student() {
        cout << "Enter Marks of 3 Subjects:\n";
        cin >> marks1 >> marks2 >> marks3;
    }

    // Function to calculate total and average
    void calculate() {
        total = marks1 + marks2 + marks3;
        average = total / 3.0;
    }

    void displayResult() {
        displayStudent(); // calling parent class function
        cout << "Marks: " << marks1 << ", " << marks2 << ", " << marks3 << endl;
        cout << "Total Marks: " << total << endl;
        cout << "Average Marks: " << average << endl;
    }
};

// Main Function
int main() {
```

```
72  int main() {
73      Result r;         // Object of Result class
74      r.calculate();    // Calculate total and average
75      r.displayResult(); // Display all data
76      return 0;
77  }
```

- **Output:**

```
Enter Name: Hunaina
Enter Age: 20
Enter Roll Number: 073
Enter Marks of 3 Subjects:
40 33 59
Name: Hunaina
Age: 20
Roll Number: 73
Marks: 40, 33, 59
Total Marks: 132
Average Marks: 44

--------------------------------
Process exited after 70.79 seconds with return value 0
Press any key to continue . . .
```

Ex 2: Design a class Person to store personal information such as name, address, and phone number. Derive a class BankAccount from Person to include account number and balance. From BankAccount, derive a class LoanAccount that includes additional fields like loan amount and interest rate. Include appropriate methods to deposit and withdraw money in BankAccount, and a method in LoanAccount to calculate the total repayable amount based on interest. Use constructor chaining and proper data encapsulation.

- **Program:**

```cpp
#include <iostream>
using namespace std;

// Base Class: Person
class Person {
protected:
    string name;
    string address;
    string phone;

public:
    // Constructor
    Person(string n, string a, string p) {
        name = n;
        address = a;
        phone = p;
    }

    // Display function
    void displayPerson() {
        cout << "Name: " << name << endl;
        cout << "Address: " << address << endl;
        cout << "Phone: " << phone << endl;
    }
};

// Derived Class: BankAccount
class BankAccount : public Person {
protected:
    int accountNumber;
    double balance;

public:
    // Constructor chaining (calling Person constructor)
    BankAccount(string n, string a, string p, int accNo, double bal)
        : Person(n, a, p) {
        accountNumber = accNo;
        balance = bal;
    }

    // Deposit function
    void deposit(double amount) {
        balance += amount;
        cout << "Deposited: " << amount << endl;
    }

    // Withdraw function
    void withdraw(double amount) {
```

```
        if (amount <= balance) {
            balance -= amount;
            cout << "Withdrawn: " << amount << endl;
        } else {
            cout << "Insufficient balance!" << endl;
        }
    }

    // Display account info
    void displayAccount() {
        displayPerson(); // calling base class function
        cout << "Account Number: " << accountNumber << endl;
        cout << "Balance: " << balance << endl;
    }
};

// Derived Class: LoanAccount
class LoanAccount : public BankAccount {
private:
    double loanAmount;
    double interestRate;

public:
    // Constructor chaining (calling BankAccount constructor)
```

```
    // Constructor chaining (calling BankAccount constructor)
    LoanAccount(string n, string a, string p, int accNo, double bal,
                double loan, double rate)
        : BankAccount(n, a, p, accNo, bal) {
        loanAmount = loan;
        interestRate = rate;
    }

    // Calculate total repayable amount
    double calculateRepayment() {
        return loanAmount + (loanAmount * interestRate / 100);
    }

    // Display loan info
    void displayLoan() {
        displayAccount();
        cout << "Loan Amount: " << loanAmount << endl;
        cout << "Interest Rate: " << interestRate << "%" << endl;
        cout << "Total Repayable Amount: " << calculateRepayment() << endl;
    }
};

// Main Function
int main() {
```

```
96      // Creating object of LoanAccount
97      LoanAccount obj("Hunaina", "Rawalpindi", "0311123789",
98                      12345, 10000, 30000, 20);
99
100     cout << "\n--- Initial Details ---\n";
101     obj.displayLoan();
102
103     cout << "\n--- After Transactions ---\n";
104     obj.deposit(2000);
105     obj.withdraw(1000);
106
107     obj.displayLoan();
108
109     return 0;
110 }
```

➢ **Output:**

```
--- Initial Details ---
Name: Hunaina
Address: Rawalpindi
Phone: 0311123789
Account Number: 12345
Balance: 10000
Loan Amount: 30000
Interest Rate: 20%
Total Repayable Amount: 36000

--- After Transactions ---
Deposited: 2000
Withdrawn: 1000
Name: Hunaina
Address: Rawalpindi
Phone: 0311123789
Account Number: 12345
Balance: 11000
Loan Amount: 30000
Interest Rate: 20%
Total Repayable Amount: 36000

--------------------------------
Process exited after 0.3096 seconds with return value 0
Press any key to continue . . .
```

Ex 3: Build a class hierarchy to represent a university structure. Start with a base class Institution that holds the name and location of the institution. Derive a class Department that includes department name and head of department. Then derive a third class Faculty which includes faculty name, subject specialization, and designation. Each class should have its own constructor and a method to display its data. Demonstrate multilevel inheritance by creating an object of the Faculty class and displaying complete details.

- **Program:**

```
#include <iostream>
using namespace std;

// Base Class
class Institution {
protected:
    string instName;
    string location;

public:
    // Constructor of Institution
    Institution(string name, string loc) {
        instName = name;
        location = loc;
    }

    // Method to display Institution data
    void displayInstitution() {
        cout << "Institution Name: " << instName << endl;
        cout << "Location: " << location << endl;
    }
};

// Derived Class 1 (inherits Institution)
```

```cpp
class Department : public Institution {
protected:
    string deptName;
    string hod;

public:
    // Constructor of Department
    Department(string name, string loc, string dName, string h)
        : Institution(name, loc) {   // Calling base class constructor
        deptName = dName;
        hod = h;
    }

    // Method to display Department data
    void displayDepartment() {
        cout << "Department Name: " << deptName << endl;
        cout << "Head of Department: " << hod << endl;
    }
};

// Derived Class 2 (inherits Department)
class Faculty : public Department {
private:
    string facultyName;
    string subject;
    string designation;

public:
    // Constructor of Faculty
    Faculty(string name, string loc, string dName, string h,
            string fName, string sub, string des)
        : Department(name, loc, dName, h) {  // Calling parent constructor
        facultyName = fName;
        subject = sub;
        designation = des;
    }

    // Method to display Faculty data
    void displayFaculty() {
        cout << "Faculty Name: " << facultyName << endl;
        cout << "Subject Specialization: " << subject << endl;
        cout << "Designation: " << designation << endl;
    }
};

// Main function
int main() {

```

```
72
73      // Creating object of Faculty class (multilevel inheritance)
74      Faculty f1("Foundation University", "Islamabad",
75                 "Computer Science", "Mr. Ahmed",
76                 "Hunaina Yasir", "Artificial Intelligence", "Lecturer");
77
78      // Display complete details
79      f1.displayInstitution();
80      f1.displayDepartment();
81      f1.displayFaculty();
82
83      return 0;
84  }
85
```

- **Output:**

```
Institution Name: Foundation University
Location: Islamabad
Department Name: Computer Science
Head of Department: Mr. Ahmed
Faculty Name: Hunaina Yasir
Subject Specialization: Artificial Intelligence
Designation: Lecturer

--------------------------------
Process exited after 0.2293 seconds with return value 0
Press any key to continue . . .
```

Ex 4: Create a base class Animal with attributes such as species and habitat. Derive a class Mammal from Animal that includes features like gestation period and isDomesticated. Further derive a class Dog from Mammal, with additional attributes like breed and age. Use appropriate constructors and methods to initialize and display the information.
Highlight the use of multilevel inheritance and how data flows through the hierarchy.

- **Program:**

```cpp
#include <iostream>
using namespace std;

// Base class
class Animal {
protected:
    string species;
    string habitat;

public:
    // Constructor of Animal
    Animal(string sp, string hab) {
        species = sp;
        habitat = hab;
    }

    // Display function
    void displayAnimal() {
        cout << "Species: " << species << endl;
        cout << "Habitat: " << habitat << endl;
    }
};

// Derived class from Animal
class Mammal : public Animal {
protected:
    int gestationPeriod;
    bool isDomesticated;

public:
    // Constructor of Mammal (calls Animal constructor)
    Mammal(string sp, string hab, int gp, bool dom)
        : Animal(sp, hab) {   // calling base class constructor
        gestationPeriod = gp;
        isDomesticated = dom;
    }
```

```
37
38      // Display function
39      void displayMammal() {
40          cout << "Gestation Period: " << gestationPeriod << " days" << endl;
41          cout << "Domesticated: " << (isDomesticated ? "Yes" : "No") << endl;
42      }
43  };
44
45  // Derived class from Mammal (Multilevel Inheritance)
46  class Dog : public Mammal {
47  private:
48      string breed;
49      int age;
50
51  public:
52      // Constructor of Dog (calls Mammal constructor)
53      Dog(string sp, string hab, int gp, bool dom, string br, int ag)
54          : Mammal(sp, hab, gp, dom) {
55          breed = br;
56          age = ag;
57      }
58
59      // Display all information
60      void displayDog() {
61          displayAnimal();   // from Animal
62          displayMammal();   // from Mammal
63          cout << "Breed: " << breed << endl;
64          cout << "Age: " << age << " years" << endl;
65      }
66  };
67
68  // Main function
69  int main() {
70      // Creating object of Dog
71      Dog d1("Canine", "Domestic/Home", 63, true, "Labrador", 3);
72
73      // Display data
74      d1.displayDog();
75
76      return 0;
77  }
```

➢ **Output:**

```
Species: Canine
Habitat: Domestic/Home
Gestation Period: 63 days
Domesticated: Yes
Breed: Labrador
Age: 3 years

--------------------------------
Process exited after 0.2311 seconds with return value 0
Press any key to continue . . .
```